

Trabajo practico Unidad 7, Base de datos.

Damian Tristant

Parte 1

1.

¿Qué es la integridad de los datos y por qué es crucial?

La integridad de los datos es el conjunto de reglas y mecanismos que garantizan que la información almacenada en una base de datos sea correcta, completa y consistente. Es crucial porque sin ella los datos se corrompen: aparecen duplicados, referencias inválidas, valores fuera de rango o registros inconsistentes. Eso provoca errores en consultas, informes incorrectos, problemas legales (por ejemplo, datos financieros), y pérdida de confianza de usuarios y aplicaciones que dependen de la BD.

Ejemplo 1: Integridad de entidad

Asegura que cada fila en una tabla sea única y pueda identificarse de forma inequívoca. Normalmente se implementa con claves primarias (PRIMARY KEY).

Ejemplo practico: Una tabla persona(dni, nombre, apellido) donde dni es PRIMARY KEY; no pueden existir dos personas con el mismo dni y no puede ser NULL.

Ejemplo 2: Integridad referencial

Garantiza que las referencias entre tablas sean válidas: un valor de clave foránea debe apuntar a una fila existente en la tabla referenciada o ser NULL si está permitido. Evita referencias “colgantes”.

Ejemplo practico: Una tabla pedido(id, cliente_id, total) donde cliente_id es FOREIGN KEY que referencia cliente(id). No se puede crear un pedido para un cliente que no existe.

Ejemplo 3: Integridad de dominio

Restringe los valores que pueden almacenarse en una columna: tipo de dato, rango, formato o reglas específicas (CHECK, NOT NULL, constraints).

Ejemplo practico: En una tabla producto(precio) se obliga precio ≥ 0 y que no sea NULL. Otro ejemplo: el campo email debe respetar un patrón básico.

2.

La consistencia en una base de datos significa que los datos siempre deben cumplir todas las reglas definidas (como las restricciones de integridad, claves foráneas, reglas de negocio, etc.) antes y después de una transacción.

Un cliente A transfiere dinero a un cliente B:

Saldo_A = 1000

Saldo_B = 500

Transferencia = 200

La operación debería:

Restar 200 del saldo de A.

Sumar 200 al saldo de B.

Durante la transacción, el sistema podría estar en un estado intermedio (por ejemplo, A ya perdió los 200, pero B todavía no los recibió).

Sin embargo, gracias a la consistencia, el sistema solo confirma (COMMIT) si el estado final cumple todas las reglas:

Ninguna cuenta queda con saldo negativo (regla del negocio).

La suma total del dinero en el sistema se mantiene igual ($1000 + 500 = 1500$ antes, $800 + 700 = 1500$ después).

Si algo falla (por ejemplo, se corta la conexión antes de acreditar en B), la transacción se revierte (ROLLBACK), y los saldos vuelven a su estado anterior. Así se evita que los datos queden “rotos” o inconsistentes.

3.

La disponibilidad de una base de datos se refiere a que el sistema esté accesible y funcionando correctamente cuando los usuarios o aplicaciones lo necesiten.

Redundancia: Tener copias de los datos en distintos servidores o ubicaciones (por ejemplo, replicación o clústeres). Si un servidor falla, otro puede continuar ofreciendo el servicio.

Tolerancia a fallos: La base de datos debe estar preparada para manejar errores de hardware, cortes de energía o fallos de red sin perder datos ni interrumpir el servicio.

Ejemplo: uso de RAID, backup automáticos o nodos secundarios activos.

Mecanismos de recuperación: Contar con copias de seguridad periódicas permite restaurar el sistema rápidamente en caso de pérdida o corrupción de datos.

Balanceo de carga: Distribuir las solicitudes entre varios servidores evita sobrecargas y mejora el rendimiento general.

Mantenimiento planificado: Actualizaciones y optimizaciones deben hacerse sin afectar el servicio (por ejemplo, con replicación en caliente o actualizaciones en nodos secundarios).

Parte 2.

1.

Una transacción de base de datos es un conjunto de operaciones que se ejecutan como una sola unidad lógica de trabajo.

Su objetivo es garantizar que los datos permanezcan correctos y consistentes, incluso si ocurren errores, cortes o múltiples accesos simultáneos.

2.

Atomicidad

Definición y propósito:

La atomicidad asegura que todas las operaciones dentro de una transacción se ejecuten por completo o no se ejecuten en absoluto. Su propósito es evitar que una transacción quede a medio realizar.

Cómo se logra:

El sistema utiliza mecanismos como logs de transacciones y comandos ROLLBACK para revertir los cambios si algo falla.

Ejemplo de violación:

En una transferencia bancaria, se descuenta dinero de la cuenta A pero ocurre un error antes de acreditar en B.

Si la transacción no fuera atómica, A pierde el dinero y B no lo recibe.

Consistencia

Definición y propósito:

Garantiza que la base de datos pase de un estado válido a otro estado válido, cumpliendo todas las restricciones de integridad y reglas del negocio.

Cómo se logra:

Aplicando restricciones (PRIMARY KEY, FOREIGN KEY, CHECK, triggers, reglas del negocio, etc.) y validando los datos antes de confirmar (COMMIT).

Ejemplo de violación:

Si se permite insertar un pedido con un cliente_id que no existe, la base de datos queda en un estado inválido.

Aislamiento

Definición y propósito:

El aislamiento garantiza que las transacciones concurrentes no interfieran entre sí.

Cada transacción debe comportarse como si fuera la única en ejecución, sin ver los cambios temporales de otras.

Cómo se logra:

A través de niveles de aislamiento (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE) y mecanismos de bloqueo o versiones múltiples (MVCC).

Ejemplo de violación:

Si una transacción lee datos que otra aún no confirmó (lectura sucia), podría basar sus cálculos en valores incorrectos.

Durabilidad

Definición y propósito:

Garantiza que, una vez confirmada una transacción (COMMIT), los cambios se mantendrán incluso si el sistema falla o se apaga.

Cómo se logra:

Guardando los cambios en almacenamiento permanente (disco) mediante logs de escritura, journaling o checkpoints.

Ejemplo de violación:

Si se confirma una transacción y luego se corta la energía, al reiniciar los datos deberían seguir igual.

Si no se cumple la durabilidad, se perderían transacciones ya confirmadas.

3.

En SQL, las sentencias BEGIN, COMMIT y ROLLBACK son las que controlan el ciclo de vida de una transacción.

Están directamente relacionadas con la propiedad de atomicidad, ya que permiten garantizar que todas las operaciones de una transacción se ejecuten como una unidad indivisible (“todo o nada”).

Parte 3.

1.

La concurrencia en bases de datos se refiere a que varios usuarios o procesos pueden acceder y modificar los datos al mismo tiempo.

Esto es común en sistemas multiusuario (por ejemplo, aplicaciones bancarias, sistemas de reservas o e-commerce), donde decenas o miles de transacciones pueden ejecutarse simultáneamente.

2.

Cuando varias transacciones se ejecutan al mismo tiempo, pueden interferir entre sí y generar resultados inconsistentes si el sistema no controla bien la concurrencia.

1. Lecturas sucias (Dirty Read)

Una transacción lee datos que otra transacción aún no confirmó (no hizo COMMIT).

Si la segunda transacción luego hace un ROLLBACK, la primera habrá usado datos que nunca existieron realmente.

Ejemplo:

Transacción A actualiza el saldo de un cliente (+100), pero aún no confirma.

Transacción B lee ese nuevo saldo.

Transacción A hace ROLLBACK.

→ El saldo leído por B era incorrecto.

Resultado:

Inconsistencia, porque se trabajó con datos “fantasma” que finalmente se descartaron.

2. Lecturas no repetibles (Non-repeatable Read)

Una transacción lee el mismo registro dos veces y obtiene valores diferentes, porque otra transacción lo modificó o eliminó entre ambas lecturas.

Ejemplo:

Transacción A consulta el saldo de una cuenta (saldo = 500).

Transacción B actualiza el saldo a 700 y confirma.

Transacción A vuelve a leer el mismo saldo y ahora obtiene 700.

Resultado:

Los resultados cambian durante la ejecución, lo que rompe la consistencia de las consultas.

3. Lecturas fantasma (Phantom Read)

Una transacción ejecuta dos veces una consulta con una condición, y en la segunda ejecución aparecen nuevas filas que antes no estaban, porque otra transacción las insertó o eliminó.

Ejemplo:

Transacción A ejecuta `SELECT * FROM pedidos WHERE total > 1000;`

Transacción B inserta un nuevo pedido con total 1500 y confirma.

Transacción A vuelve a ejecutar la misma consulta y ahora aparece una fila más.

Resultado:

Los resultados cambian aunque A no haya modificado nada, lo que puede afectar informes o cálculos agregados.

4. Actualizaciones perdidas (Lost Update)

Dos transacciones leen el mismo dato y luego lo actualizan, pero la última sobrescribe el cambio de la primera, haciendo que uno de los resultados se pierda.

Ejemplo:

Transacción A y B leen stock = 10.

A resta 2 unidades y guarda (stock = 8).

B resta 1 y guarda (stock = 9).

→ El cambio de A se pierde, porque B guardó encima.

Resultado:

Pérdida de información y resultados incorrectos.

3.

- Los niveles de aislamiento definen cómo y cuándo los cambios realizados por una transacción se vuelven visibles para otras transacciones concurrentes. Están diseñados para equilibrar la coherencia de los datos (evitando problemas de concurrencia) con el rendimiento.

Los cuatro niveles de aislamiento estándar (definidos por el estándar SQL) y los problemas de concurrencia que previenen o permiten son:

Nivel de Aislamiento - Prevención de Problemas - Problemas Permitidos

READ UNCOMMITTED - Ninguno (el más bajo) - Lecturas Sucias (Dirty Reads), Lecturas No Repetibles, Filas Fantasma (Phantom Reads).

READ COMMITTED - Lecturas Sucias (Dirty Reads) - Lecturas No Repetibles, Filas Fantasma (Phantom Reads).

REPEATABLE READ - Lecturas Sucias, Lecturas No Repetibles - Filas Fantasma (Phantom Reads).

SERIALIZABLE - Lecturas Sucias, Lecturas No Repetibles, Filas Fantasma (el más alto) - Ninguno (máximo aislamiento).

Lectura Sucia (Dirty Read): Una transacción lee datos que fueron modificados por otra transacción que aún no ha sido confirmada (commit). Si la segunda transacción luego falla y se revierte (rollback), la primera transacción leyó datos que nunca existieron realmente.

Lectura No Repetible (Non-Repeatable Read): Una transacción lee la misma fila dos veces y obtiene valores diferentes porque otra transacción modificó y confirmó el cambio entre las dos lecturas.

Fila Fantasma (Phantom Read): Una transacción ejecuta una consulta de rango (ej. WHERE salario > 50000) dos veces y obtiene un conjunto de filas diferente la segunda vez porque otra transacción insertó o eliminó filas que caen dentro del rango entre las dos ejecuciones.

- Existe una relación inversa entre el nivel de aislamiento y el rendimiento de la base de datos:

Aislamiento Más Alto (Ej: SERIALIZABLE):

Proporciona la mayor consistencia y coherencia de los datos (evita todos los problemas).

Reduce el rendimiento porque requiere un uso más intensivo de bloqueos (locks). Estos bloqueos son más restrictivos y se mantienen por más tiempo, lo que aumenta la probabilidad de esperas (waits) y bloqueos mutuos (deadlocks) entre transacciones.

Aislamiento Más Bajo (Ej: READ UNCOMMITTED):

Ofrece el peor nivel de consistencia (permite todos los problemas de concurrencia).

Maximiza el rendimiento porque utiliza la menor cantidad de bloqueos (o ninguno en lecturas), permitiendo que las transacciones se ejecuten con mayor rapidez y menos interferencia.

- El nivel de aislamiento se establece típicamente al comienzo de una transacción o se define de forma predeterminada a nivel de sesión.

La sintaxis SQL estándar es:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

4.

Bloqueo:

El bloqueo es el mecanismo más común para controlar la concurrencia.

Consiste en restringir el acceso a ciertos datos mientras una transacción los está usando, para evitar que otras los modifiquen al mismo tiempo.

Cuando una transacción obtiene un “lock”, las demás deben esperar hasta que se libere para acceder a esos mismos datos.

Tipos de bloqueo:

- Bloqueo compartido (Shared Lock, S):

Permite lecturas simultáneas, pero bloquea las escrituras.

Varias transacciones pueden leer el mismo registro, pero ninguna puede modificarlo hasta que se liberen los bloqueos.

- Bloqueo exclusivo (Exclusive Lock, X):

Solo una transacción puede tenerlo.

Impide que otras transacciones lean o modifiquen el dato hasta que el bloqueo se libere (normalmente al hacer COMMIT o ROLLBACK).

- Bloqueos de intención (Intention Locks):

Indican la intención de obtener un bloqueo más específico en un nivel inferior (por ejemplo, en una fila dentro de una tabla).

Ayudan al gestor de la base de datos a manejar múltiples niveles de bloqueo (tabla, página, fila).

Ejemplo entre usuario A y B:

Usuario A está actualizando el saldo de una cuenta.

Usuario B intenta hacer lo mismo al mismo tiempo.

Cuando A ejecuta el UPDATE, el sistema coloca un bloqueo exclusivo (X) sobre la fila.

B intenta acceder a la misma fila, pero debe esperar a que A confirme (COMMIT) o cancele (ROLLBACK).

Cuando A termina, el bloqueo se libera y B puede continuar con la operación.

Así se evitan actualizaciones perdidas o lecturas inconsistentes.

Ordenamiento por marca de tiempo:

Este método asigna a cada transacción un sello de tiempo (timestamp) cuando comienza.

El sistema usa esos tiempos para determinar el orden en que deben aplicarse las operaciones, como si se hubieran ejecutado una detrás de otra.

Si una transacción intenta modificar un dato más “antiguo” que otro cambio ya realizado por una transacción más reciente, se cancela o se reinicia.

Control de concurrencia de múltiples versiones:

Crea múltiples versiones de los datos para permitir lecturas y escrituras concurrentes sin bloquear.

Cuando una transacción lee un dato, ve la versión más reciente que era válida al momento en que comenzó, sin importar si otros lo están modificando.

Cuando una transacción actualiza un dato, crea una nueva versión del registro. Las demás transacciones que comenzaron antes siguen viendo la versión vieja hasta que terminen.

5.

Un bloqueo mutuo, o deadlock, ocurre cuando dos o más transacciones quedan esperando indefinidamente porque cada una posee un recurso (bloqueo) que la otra necesita para continuar.

En otras palabras, ninguna puede avanzar ni liberar sus bloqueos, generando un “empate” que detiene el progreso del sistema.

Métodos para resolver un deadlock:

Detección automática por el sistema:

Los gestores de bases de datos modernos (como MySQL, PostgreSQL o SQL Server) tienen algoritmos que detectan ciclos de espera entre transacciones.

Cuando se detecta un deadlock, el sistema elige una transacción como “víctima” (generalmente la más corta o la menos costosa) y la revierte (ROLLBACK).

La otra transacción continúa normalmente.

Prevención mediante orden de acceso:

Diseñar las transacciones para que accedan a los recursos en el mismo orden.

Por ejemplo, si todas las transacciones primero bloquean fila1 y después fila2, nunca se producirá un ciclo de espera.

Uso de tiempos de espera (timeout):

Si una transacción no puede obtener un bloqueo después de cierto tiempo, se cancela automáticamente.

Esto evita esperas infinitas.

Revisión del diseño y aislamiento:

Mantener las transacciones lo más cortas posible.

Usar el nivel de aislamiento adecuado para reducir bloqueos innecesarios.

Revisar la lógica de actualización para evitar accesos cruzados.

Parte 4

1.

a) Sistemas Bancarios:

En la banca, cada operación —como transferencias, depósitos o pagos— se maneja mediante transacciones.

Si una transacción no se ejecuta de manera atómica, podrían producirse errores como dinero debitado sin ser acreditado.

Las propiedades ACID garantizan que los movimientos sean coherentes, incluso frente a fallas de red o cortes de energía.

b) Comercio Electrónico (E-commerce):

Cuando un cliente realiza una compra, la base de datos debe actualizar el stock, registrar el pedido y procesar el pago.

Todas esas operaciones deben ocurrir juntas o ninguna, para evitar inconsistencias como vender productos que ya no hay disponibles.

El uso de transacciones asegura la coherencia entre inventario, pagos y órdenes.

c) Sistemas Hospitalarios o Clínicos:

En hospitales, la información del paciente (historial médico, tratamientos, medicación) debe actualizarse con precisión.

Si una transacción falla a mitad de camino, podría provocar errores en diagnósticos o medicación, afectando la seguridad del paciente.

Por eso, los sistemas hospitalarios utilizan transacciones para mantener datos exactos y confiables.

2.

Pregunta 1.

Control de concurrencia pesimista:

Supone que los conflictos van a ocurrir, por lo que bloquea los recursos antes de modificarlos.

Utiliza bloqueos (locks) para evitar que otras transacciones accedan a los mismos datos hasta que la actual termine.

Es ideal cuando hay alta probabilidad de conflicto, como en sistemas bancarios o de inventario donde muchos usuarios actualizan los mismos registros.

Ventaja: evita conflictos de forma segura.

Desventaja: puede reducir el rendimiento y causar esperas o deadlocks.

Control de concurrencia optimista:

Supone que los conflictos son poco frecuentes, por lo que permite el acceso libre a los datos.

Al finalizar la transacción, el sistema verifica si alguien más modificó los datos en ese tiempo. Si detecta un conflicto, rechaza o reinicia la transacción.

Es ideal para entornos donde predominan las lecturas y las actualizaciones son raras, como sistemas de reportes o aplicaciones analíticas.

Ventaja: mayor rendimiento y menos bloqueos.

Desventaja: posibles reintentos en caso de conflicto.

Fuentes consultadas:

Documentación oficial de PostgreSQL y MySQL.

AWS Database Blog – “How databases handle concurrent transactions”.

Pregunta 2.

Implementación de las propiedades ACID:

Atomicidad:

PostgreSQL utiliza un Write-Ahead Log (WAL), donde registra todas las operaciones antes de aplicarlas.

Si ocurre un fallo, el sistema usa este log para deshacer o rehacer las operaciones pendientes, garantizando el “todo o nada”.

Consistencia:

Se mantiene a través de restricciones (PRIMARY KEY, FOREIGN KEY, CHECK), triggers y reglas de negocio.

PostgreSQL no permite confirmar una transacción que deje la base en un estado inválido.

Aislamiento:

Implementado mediante MVCC (Multi-Version Concurrency Control).

Esto permite que las lecturas nunca bloqueen escrituras y viceversa.

Cada transacción ve una “fotografía” consistente de los datos al momento en que comenzó.

Durabilidad:

Una vez confirmado el COMMIT, PostgreSQL asegura que los cambios están guardados físicamente en el log (WAL).

En caso de caída del sistema, los datos confirmados se recuperan automáticamente.

Niveles de aislamiento disponibles en PostgreSQL:

READ UNCOMMITTED (se comporta igual que READ COMMITTED)

READ COMMITTED ( nivel predeterminado)

REPEATABLE READ

SERIALIZABLE

Fuentes consultadas:

PostgreSQL Documentation: Transactions and Concurrency Control (versión 16).

PostgreSQL Wiki – MVCC Internals Explained.

Redgate Blog – “How PostgreSQL handles ACID compliance” (2022).